Implementation Tasks

API Key: `t2On0w9hkjQNrfnKEaO7FhsVrfPLXZS2`
Base URL: `https://interview-api.memtime-demo.deno.net/api/v1`

Task 1: Display Clients, Projects, and Tasks Hierarchically

Implement a UI component that displays data in a tree structure:

1. **Fetch all clients** using `GET /clients`
2. **For each client**, fetch projects using `GET /clients/:id/projects`
3. **For each project**, fetch tasks using `GET /projects/:id/tasks`
4. **Display hierarchy:**

```
└─ Client 1
   └─ Project 1
      └─ Task 1
      └─ Task 2
   └─ Project 2
└─ Client 2
```

Note: Use pagination parameters (`limit`, `offset`) as needed. Refer to `API_V1_DOCUMENTATION.md` for full details.

Task 2: Display Time Entries

Implement a list view for time entries per task:

1. **Fetch user's time entries** using `GET /time-entries`
2. **Display columns:** ID, Task ID, Comment, Start, End, Created At
3. **Support pagination** with query parameters

Task 3: Create or Update Time Entry

Implement a form with create and update functionality:

Create New Time Entry

- **Endpoint:** `POST /time-entries`
- **Required fields:** `taskId`, `comment`, `start` (ISO 8601), `end` (ISO 8601)
- **Display:** Success message with new entry ID

Update Existing Time Entry

- **Endpoint:** `PUT /time-entries/:id`
- **Required fields:** `taskId`, `comment`, `start` (ISO 8601), `end` (ISO 8601)
- **Display:** Success message with updated entry

Error Handling

- Display meaningful error messages for invalid input or API errors
- Handle 404 errors (entry not found)
- Handle 400 errors (invalid JSON body)

Implementation Notes

- **Authentication:** Include `Authorization: Bearer <API_KEY>` header in all requests
- **Rate Limiting:** API allows 15 requests per 60 seconds
- **Response Format:** All responses are JSON with pretty-printing enabled
- **Error Handling:** See API_V1_DOCUMENTATION.md for error response formats

Notes

While this is not only a front-end focused task, please consider the following when implementing your solution:

- Imagine this solution will be used by paying customers next week. Please deliver something you would be proud to put your name on.
- How a real user would interact with it, whether the UX encourages correct and easy time entry, and what you would improve if this were part of a larger product.